

计算机组成原理创新实验

实验名称：8 位 AI 运算器设计-第二周

学号：2410668

姓名：吕昊远

班次：张金、董前琨

一、实验任务

1. 3×3 卷积单元

模块代码：

```
1 module conv3x3_unit (
2     input wire [71:0] pixels, // 9个像素
3     input wire [71:0] weights, // 9个权重
4     output wire [7:0] conv_out
5 );
6     wire [7:0] p[0:8], w[0:8], mult_res[0:8];
7     genvar i;
8     generate
9         for (i = 0; i < 9; i = i + 1) begin : MAC
10             assign p[i] = pixels[(i*8+7) : i*8];
11             assign w[i] = weights[(i*8+7) : i*8];
12             mult_8 u_m (.a(p[i]), .b(w[i]), .p(mult_res[i]));
13         end
14     endgenerate
15     // 串行相加
16     wire [7:0] sum1, sum2, sum3, sum4, sum5, sum6, sum7, sum8;
17     adder_8 u_add1 (.a(mult_res[0]), .b(mult_res[1]), .sum(sum1));
18     adder_8 u_add2 (.a(sum1), .b(mult_res[2]), .sum(sum2));
19     adder_8 u_add3 (.a(sum2), .b(mult_res[3]), .sum(sum3));
20     adder_8 u_add4 (.a(sum3), .b(mult_res[4]), .sum(sum4));
21     adder_8 u_add5 (.a(sum4), .b(mult_res[5]), .sum(sum5));
22     adder_8 u_add6 (.a(sum5), .b(mult_res[6]), .sum(sum6));
23     adder_8 u_add7 (.a(sum6), .b(mult_res[7]), .sum(sum7));
24     adder_8 u_add8 (.a(sum7), .b(mult_res[8]), .sum(sum8));
25
26     assign conv_out = sum8;
27 endmodule
```

2. 2x2 池化单元

模块代码:

```
1 module pool2x2_unit (
2     input wire [7:0] p1, p2, p3, p4,
3     output wire [7:0] pool_out
4 );
5     wire [9:0] sum = p1 + p2 + p3 + p4;
6     div_4 u_div (.in_data(sum), .out_data(pool_out));
7 endmodule
```

3. 全链路系统集成

模块代码:

```
1 module cnn_core_top (
2     input wire [199:0] img_5x5, // 25个8位像素
3     input wire [71:0] weight_3x3, // 9个8位权重
4     output wire [31:0] out_2x2 // 4个8位特征输出
5 );
6     wire [7:0] img [0:24];
7     genvar k;
8     generate
9         for (k = 0; k < 25; k = k + 1) begin : UNPACK
10             assign img[k] = img_5x5[(k*8+7) : k*8];
11         end
12     endgenerate
13
14     wire [7:0] conv_map [0:8], relu_map [0:8], final_out [0:3];
15
16     // 对9个3x3窗口进行卷积
17     wire [71:0] win_0 = {img[12],img[11],img[10], img[7],img[6],img[5], img[2],img
18         [1],img[0]};
19     wire [71:0] win_1 = {img[13],img[12],img[11], img[8],img[7],img[6], img[3],img
20         [2],img[1]};
21     wire [71:0] win_2 = {img[14],img[13],img[12], img[9],img[8],img[7], img[4],img
22         [3],img[2]};
23     wire [71:0] win_3 = {img[17],img[16],img[15], img[12],img[11],img[10], img[7],
24         img[6],img[5]};
25     wire [71:0] win_4 = {img[18],img[17],img[16], img[13],img[12],img[11], img[8],
26         img[7],img[6]};
27     wire [71:0] win_5 = {img[19],img[18],img[17], img[14],img[13],img[12], img[9],
28         img[8],img[7]};
29     wire [71:0] win_6 = {img[22],img[21],img[20], img[17],img[16],img[15], img[12],
30         img[11],img[10]};
31     wire [71:0] win_7 = {img[23],img[22],img[21], img[18],img[17],img[16], img[13],
32         img[12],img[11]};
33     wire [71:0] win_8 = {img[24],img[23],img[22], img[19],img[18],img[17], img[14],
34         img[13],img[12]};
```

```

27 // 卷积单元
28 conv3x3_unit u_conv_0 (.pixels(win_0), .weights(weight_3x3), .conv_out(conv_map
    [0]));
29 conv3x3_unit u_conv_1 (.pixels(win_1), .weights(weight_3x3), .conv_out(conv_map
    [1]));
30 conv3x3_unit u_conv_2 (.pixels(win_2), .weights(weight_3x3), .conv_out(conv_map
    [2]));
31 conv3x3_unit u_conv_3 (.pixels(win_3), .weights(weight_3x3), .conv_out(conv_map
    [3]));
32 conv3x3_unit u_conv_4 (.pixels(win_4), .weights(weight_3x3), .conv_out(conv_map
    [4]));
33 conv3x3_unit u_conv_5 (.pixels(win_5), .weights(weight_3x3), .conv_out(conv_map
    [5]));
34 conv3x3_unit u_conv_6 (.pixels(win_6), .weights(weight_3x3), .conv_out(conv_map
    [6]));
35 conv3x3_unit u_conv_7 (.pixels(win_7), .weights(weight_3x3), .conv_out(conv_map
    [7]));
36 conv3x3_unit u_conv_8 (.pixels(win_8), .weights(weight_3x3), .conv_out(conv_map
    [8]));
37
38 // relu层
39 relu u_relu_0 (.in_data(conv_map[0]), .out_data(relu_map[0]));
40 relu u_relu_1 (.in_data(conv_map[1]), .out_data(relu_map[1]));
41 relu u_relu_2 (.in_data(conv_map[2]), .out_data(relu_map[2]));
42 relu u_relu_3 (.in_data(conv_map[3]), .out_data(relu_map[3]));
43 relu u_relu_4 (.in_data(conv_map[4]), .out_data(relu_map[4]));
44 relu u_relu_5 (.in_data(conv_map[5]), .out_data(relu_map[5]));
45 relu u_relu_6 (.in_data(conv_map[6]), .out_data(relu_map[6]));
46 relu u_relu_7 (.in_data(conv_map[7]), .out_data(relu_map[7]));
47 relu u_relu_8 (.in_data(conv_map[8]), .out_data(relu_map[8]));
48
49 // 池化输出
50 pool2x2_unit u_pool_0 (.p1(relu_map[0]), .p2(relu_map[1]), .p3(relu_map[3]), .
    p4(relu_map[4]), .pool_out(final_out[0]));
51 pool2x2_unit u_pool_1 (.p1(relu_map[1]), .p2(relu_map[2]), .p3(relu_map[4]), .
    p4(relu_map[5]), .pool_out(final_out[1]));
52 pool2x2_unit u_pool_2 (.p1(relu_map[3]), .p2(relu_map[4]), .p3(relu_map[6]), .
    p4(relu_map[7]), .pool_out(final_out[2]));
53 pool2x2_unit u_pool_3 (.p1(relu_map[4]), .p2(relu_map[5]), .p3(relu_map[7]), .
    p4(relu_map[8]), .pool_out(final_out[3]));
54
55 assign out_2x2 = {final_out[3], final_out[2], final_out[1], final_out[0]};

```

在第二周的部分，我们将先前实现的运算模块集成到了卷积层与池化层，并依此实现了全链路的运算模块。由于数据较少，此处手动写出每层处理的逻辑与相互之间的连接，卷积核大小为 3x3，在 5x5 的像素矩阵中会滑动 9 次，得到 9 个卷积结果，再分别连接 relu 层的激活函数进行激活，最终通过平均池化层计算出 2x2 大小的特征值矩阵。

接下来将编写仿真文件对上述代码进行验证，同时生成仿真波形图观察是否正确。

二、仿真波形验证

仿真代码如下：

```
1 // w2_tb.v
2 `timescale 1ns / 1ps
3
4 module tb_week2();
5     reg [199:0] img_in; // 25x8
6     reg [71:0] weight_in;
7     wire [31:0] cnn_out;
8
9     cnn_core_top u_top (
10         .img_5x5(img_in),
11         .weight_3x3(weight_in),
12         .out_2x2(cnn_out)
13     );
14
15     initial begin
16         // Test1: 标准5x5灰度矩阵，像素全10，权重全1
17         weight_in = {9{8'd1}};
18         img_in = {25{8'd10}};
19         #100;
20         $display("Test1 | weight: %h | img: %h | out: %d, %d, %d, %d",
21             weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
22             cnn_out[7:0]);
23
24         // Test2: 孤点图像，仅像素0为20，其余为0，权重全1
25         img_in = 200'b0;
26         img_in[7:0] = 8'd20;
27         #100;
28         $display("Test2 | weight: %h | img: %h | out: %d, %d, %d, %d",
29             weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
30             cnn_out[7:0]);
31
32         // Test3: 零输入，图像全0，权重全1
33         img_in = 200'b0;
34         #100;
35         $display("Test3 | weight: %h | img: %h | out: %d, %d, %d, %d",
36             weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
37             cnn_out[7:0]);
38
39         // Test4: 低亮度，像素全4，权重全1
40         img_in = {25{8'd4}};
41         #100;
42         $display("Test4 | weight: %h | img: %h | out: %d, %d, %d, %d",
43             weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
44             cnn_out[7:0]);
45
46         // Test5: 提升权重为2，像素全10
```

```

43     img_in      = {25{8'd10}};
44     weight_in = {9{8'd2}};
45     #100;
46     $display("Test5 | weight: %h | img: %h | out: %d, %d, %d, %d",
47             weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
48             cnn_out[7:0]);
49
50     // Test6: 对角线, 对角线的像素为10, 权重全1
51     img_in = 200'b0;
52     img_in[7:0]      = 8'd10;
53     img_in[55:48]    = 8'd10;
54     img_in[103:96]   = 8'd10;
55     img_in[151:144]=8'd10;
56     img_in[199:192]=8'd10;
57     weight_in = {9{8'd1}};
58     #100;
59     $display("Test6 | weight: %h | img: %h | out: %d, %d, %d, %d",
60             weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
61             cnn_out[7:0]);
62
63     // Test7: 权重全0, 像素全255,
64     img_in      = {25{8'd255}};
65     weight_in = 72'b0;
66     #100;
67     $display("Test7 | weight: %h | img: %h | out: %d, %d, %d, %d",
68             weight_in, img_in, cnn_out[31:24], cnn_out[23:16], cnn_out[15:8],
69             cnn_out[7:0]);
70
71     $finish;
72 end
73 endmodule

```

仿真波形如下:

输入图像矩阵 (5×5):

$$\begin{pmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 90 & 90 \\ 90 & 90 \end{pmatrix}$$

Test 2

卷积核权重矩阵 (3×3):

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

输入图像矩阵 (5×5):

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 20 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 0 & 0 \\ 0 & 5 \end{pmatrix}$$

Test 3

卷积核权重矩阵 (3×3):

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

输入图像矩阵 (5×5):

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

Test 4

卷积核权重矩阵 (3×3):

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

输入图像矩阵 (5×5):

$$\begin{pmatrix} 4 & 4 & 4 & 4 & 4 \\ 4 & 4 & 4 & 4 & 4 \\ 4 & 4 & 4 & 4 & 4 \\ 4 & 4 & 4 & 4 & 4 \\ 4 & 4 & 4 & 4 & 4 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 36 & 36 \\ 36 & 36 \end{pmatrix}$$

Test 5

卷积核权重矩阵 (3×3):

$$\begin{pmatrix} 2 & 2 & 2 \\ 2 & 2 & 2 \\ 2 & 2 & 2 \end{pmatrix}$$

输入图像矩阵 (5×5):

$$\begin{pmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \\ 10 & 10 & 10 & 10 & 10 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 180 & 180 \\ 180 & 180 \end{pmatrix}$$

Test 6

卷积核权重矩阵 (3×3):

$$\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

输入图像矩阵 (5×5):

$$\begin{pmatrix} 10 & 0 & 0 & 0 & 0 \\ 0 & 10 & 0 & 0 & 0 \\ 0 & 0 & 10 & 0 & 0 \\ 0 & 0 & 0 & 10 & 0 \\ 0 & 0 & 0 & 0 & 10 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 25 & 20 \\ 20 & 25 \end{pmatrix}$$

Test 7

卷积核权重矩阵 (3×3):

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

输入图像矩阵 (5×5):

$$\begin{pmatrix} 255 & 255 & 255 & 255 & 255 \\ 255 & 255 & 255 & 255 & 255 \\ 255 & 255 & 255 & 255 & 255 \\ 255 & 255 & 255 & 255 & 255 \\ 255 & 255 & 255 & 255 & 255 \end{pmatrix}$$

输出特征矩阵 (2×2):

$$\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$

经计算，上述几组测试数据结果完全正确，模块代码的计算逻辑准确无误。同时，观察图像的特点与计算得出特征矩阵，我们也能发现：经过一系列处理，模型对有效特征的提取效果显著。例如 Test6 的对角线图像矩阵，经过处理后的特征矩阵依然保留了对角线处数值较高的特征（25，20，20，25），是算法效果的直观体现。

截至到这里，我们已经完成了最为基础的计算模块，可以完整而正确地实现要求的功能。在下一周的部分，将在此基础上进行改进，对 LUT 资源占用与计算时延进行优化，以提升整体的计算效率。